

Exercício 01 — Classes que se relacionam

Aquecimento em TypeScript antes de entrar em arquitetura

Prof. Maikon Bueno

O que você vai construir

Uma pequena biblioteca com **duas classes que se conhecem**:

Livro

Sabe qual é o seu autor.

Autor

Sabe quais livros escreveu.

E mais três **funções soltas** que trabalham sobre uma lista de livros.

Antes de começar — criando o projeto

1. Criar a pasta e instalar

```
mkdir biblioteca
cd biblioteca
npm init -y
npm install --save-dev \
  typescript@5 ts-node @types/node
npx tsc --init
mkdir src
```

3. Rodar o programa

```
npx ts-node src/main.ts
```

2. Ajustar o `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
    "strict": true
  }
}
```

O `@5` é obrigatório: o `ts-node` **não funciona** com o TypeScript 7. E sem o `tsconfig.json` ele falha com `Unknown file extension ".ts"`.

O problema

Suponha que o autor seja apenas um texto dentro do livro:

```
const livro = new Livro('Dom Casmurro', 'Machado de Assis');
```

Se o nome vier escrito diferente

```
'Machado de Assis'
```

```
'machado de assis'
```

Para o programa, são dois autores.

E não dá para perguntar

"quais livros o Machado escreveu?"

O texto não sabe responder. Ele é só um texto.

A ideia: uma classe aponta para a outra

O autor deixa de ser texto e vira **objeto**:

```
const machado = new Autor('Machado de Assis');  
const livro = new Livro('Dom Casmurro', 1899, machado);
```

Agora `livro.autor` é um **Autor** de verdade — e existe um só Machado na memória, mesmo que ele tenha escrito dez livros.

Mão dupla

Do livro para o autor

```
livro.autor.nome
```

Todo livro tem **um** autor.

Do autor para os livros

```
machado.livros
```

Todo autor tem **vários** livros — por isso, uma lista.

É esta ligação nos dois sentidos que se chama **relacionamento entre classes**. Cada uma guarda uma ponta.

Os arquivos

Uma classe por arquivo:

```
biblioteca/  
└─ src/  
    └─ autor.ts      <- a classe Autor  
    └─ livro.ts      <- a classe Livro  
    └─ relatorio.ts  <- funções soltas (nenhuma classe aqui)  
    └─ main.ts       <- o programa que usa tudo
```

Faça **nesta ordem**. O `livro.ts` precisa do `autor.ts` já escrito.

Passo 1 — `src/autor.ts`

```
import { Livro } from './livro';

export class Autor {
  nome: string;
  livros: Livro[];

  constructor(nome: string) {
    this.nome = nome;
    this.livros = []; // nasce sem nenhum livro
  }

  adicionarLivro(livro: Livro): void {
    this.livros.push(livro);
  }
}
```

Acrescente também `quantidadeDeLivros(): number`, devolvendo o tamanho da lista.

Passo 2 — `src/livro.ts`

```
import { Autor } from './autor';

export class Livro {
  titulo: string;
  ano: number;
  autor: Autor;

  constructor(titulo: string, ano: number, autor: Autor) {
    this.titulo = titulo;
    this.ano = ano;
    this.autor = autor;

    autor.adicionarLivro(this); // <- a linha importante
  }
}
```

Acrescente `descricao(): string`, devolvendo `"Dom Casmurro (1899), de Machado de Assis"`.

A linha importante

```
autor.adicionarLivro(this);
```

O que ela diz

*"autor, se registre: **eu** sou um livro seu".*

Aqui `this` é o livro que está nascendo agora.

Sem ela

O livro conheceria o autor, mas o autor **não** conheceria o livro.

A mão dupla ficaria pela metade.

Como ela está dentro do `constructor`, é **impossível** criar um livro e esquecer de registrá-lo no autor.

Passo 3 — `src/relatorio.ts`

Três funções **fora de qualquer classe**. Todas recebem uma lista de livros:

```
import { Livro } from './livro';  
  
export function titulos(livros: Livro[]): string[] { }
```

devolve só os títulos, numa lista de textos

```
export function publicadosDepoisDe(livros: Livro[], ano: number): Livro[] { }
```

devolve os livros com ano **maior** que o recebido

```
export function maisAntigo(livros: Livro[]): Livro { }
```

devolve o livro de menor ano

Use `for...of` nas três: crie uma lista vazia, vá dando `push` e devolva no fim.

Passo 4 — `src/main.ts`

```
import { Autor } from './autor';
import { Livro } from './livro';
import { titulos, publicadosDepoisDe, maisAntigo } from './relatorio';

const machado = new Autor('Machado de Assis');
const orwell = new Autor('George Orwell');

const acervo: Livro[] = [];
acervo.push(new Livro('Dom Casmurro', 1899, machado));
acervo.push(new Livro('Memórias Póstumas de Brás Cubas', 1881, machado));
acervo.push(new Livro('1984', 1949, orwell));
acervo.push(new Livro('A Revolução dos Bichos', 1945, orwell));
```

Agora **você** escreve o resto — está na próxima página.

Passo 4 — o que imprimir

Do livro para o autor

1. percorra o `acervo` com `for...of`
2. imprima a `descricao()` de cada livro

Do autor para os livros

3. imprima quantos livros o Machado escreveu
4. percorra `machado.livros` e imprima o título de cada um
5. repita para o Orwell

Usando as funções

6. imprima `titulos(acervo)`
7. imprima os títulos dos livros publicados **depois de 1900**
8. imprima a descrição do livro **mais antigo**

No item 7 dá para encaixar uma função dentro da outra:

```
titulos(publicadosDepoisDe(acervo, 1900))
```

Saída esperada

```
--- Descrição de cada livro ---  
Dom Casmurro (1899), de Machado de Assis  
Memórias Póstumas de Brás Cubas (1881), de Machado de Assis  
1984 (1949), de George Orwell  
A Revolução dos Bichos (1945), de George Orwell  
  
--- Livros de cada autor ---  
Machado de Assis escreveu 2 livro(s):  
- Dom Casmurro  
- Memórias Póstumas de Brás Cubas  
George Orwell escreveu 2 livro(s):  
- 1984  
- A Revolução dos Bichos  
  
--- Relatórios ---  
Publicados depois de 1900: [ '1984', 'A Revolução dos Bichos' ]  
Mais antigo: Memórias Póstumas de Brás Cubas (1881), de Machado de Assis
```

Erros comuns

Esquecer o `autor.adicionarLivro(this)`

```
machado.livros.length // 0
```

O autor fica sem livro nenhum.

Escrever a extensão no import

```
import { Autor } from './autor.ts'; // ERRO
import { Autor } from './autor';    // certo
```

Esquecer de iniciar a lista

```
constructor(nome: string) {
  this.nome = nome;
  // faltou this.livros = [];
}
```

O `push` explode com `undefined`.

Esquecer o `export`

Sem ele, o outro arquivo não enxerga a classe.